EventLink

Databases

Team members

Nuriel Taha
Amine Yesfi
Yilikal Sarka
Bekzat Arystanov

Course: Databases
Instructor: Peter Baumann

Intended functionality

EventLink is a web platform designed to make the process of buying and managing tickets to seated events as smooth and enjoyable as possible. Customers can easily discover upcoming events through a search and filter system, browse by keyword, date, or category, and view details such as the description, venue information, showtimes, and pricing tiers. An interactive seating map allows users to check availability in real time and compare prices for different sections. Consequently, they can select seats that match their personal preferences, whether they want front-row spots or more affordable alternatives. Seats are temporarily reserved once chosen, giving the customer a secure window to complete the payment without having to worry about losing their selection. Upon a successful purchase, customers get an immediate delivery of e-tickets, in the form or QR codes. These codes can be saved either as a PDF, stored in the user's account, or added directly to the wallet for convenience. If an event is sold out, customers are offered a waitlist feature. Customers can sign up for specific seat types and receive automatic notifications if tickets are released. They will then have a limited pickup window to confirm their tickets.

ER diagram

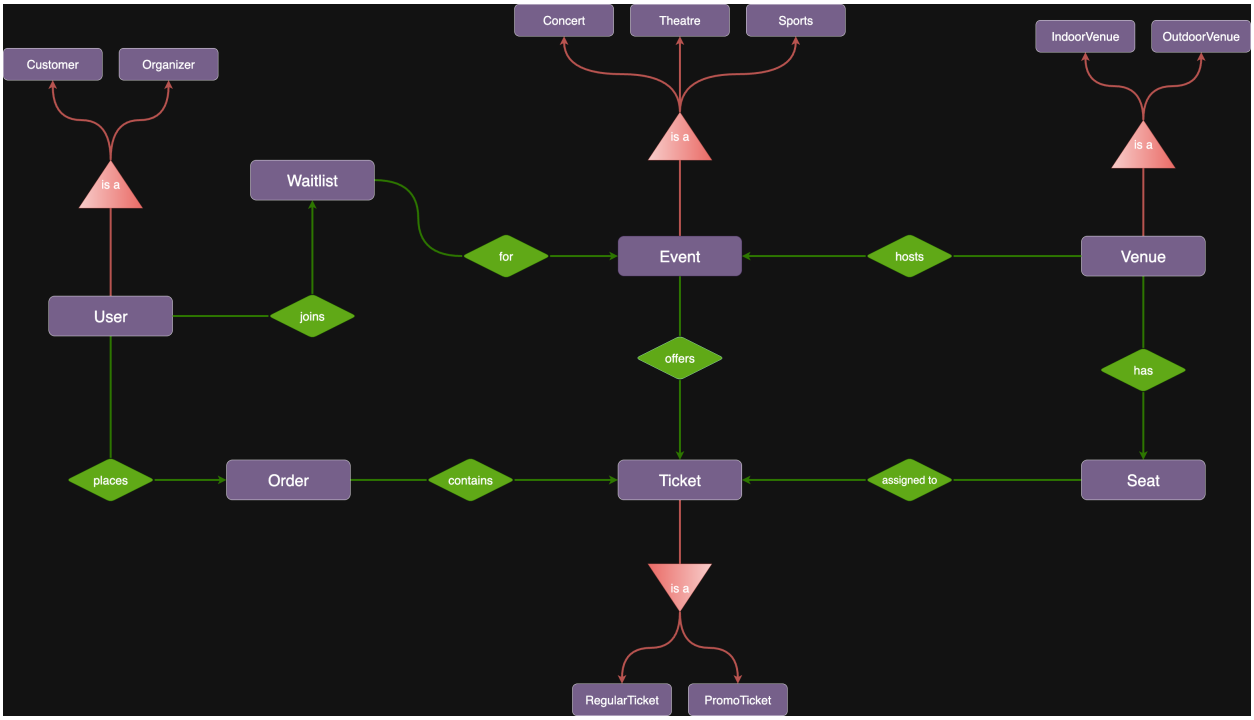

Figure 1: ER model diagram without attributes

List of User Interactions

1. **Browsing events**
    - User sees a catalogue of upcoming events.
    - Can filter by date, category, or keyword.
    - $\Rightarrow$ quickly find events of interest.
2. **Viewing event details**
    - Event page with description, venue, showtimes, and pricing.
    - Interactive seating map with real-time availability.
    - $\Rightarrow$ informed decision-making.
3. **Seat selection and checkout**
    - User selects available seats (temporarily held during checkout).
    - Proceeds to payment flow.
    - Successful payment $\Rightarrow$ confirmation & e-ticket delivery.
    - Invalid payment or timeout $\Rightarrow$ seats are released, error shown.
4. **Ticket management**
    - Access purchased tickets in account.
    - Download as PDF or add to mobile wallet.
    - Attempting to access others' tickets $\Rightarrow$ blocked with error message.
5. **Waitlist handling**
    - If sold out, user can join a waitlist with seat preferences.
    - Notification sent when tickets become available.
    - Expired offers $\Rightarrow$ passed to next person in line.
6. **Account and history**
    - User logs in to view past orders, active tickets, and waitlist entries.
    - Incorrect login credentials $\Rightarrow$ error feedback.
7. **Refunds and cancellations**
    - User may request refunds or cancel orders.
    - Valid request $\Rightarrow$ seat returned to inventory, refund processed.
    - Invalid request (e.g. refund deadline passed) $\Rightarrow$ denied with explanation.

Mapping approach

Entities

Each entity becomes its own table with a surrogate primary key: User(user_id), Venue(venue_id), Seat(seat_id), Event(event_id), Ticket(ticket_id), Order(order_id), Waitlist(entry_id). We store authentication as password (hashed of course). Status-like attributes use ENUMMs for readability.

Relationships

One-to-many relationships are realized by placing the foreign key on the ”many” side:

- **User - places - Order:** Order.user_id → User.user_id.
- **Venue - hosts - Event:** Event.venue_id → Venue.venue_id.
- **Venue - has - Seat:** Seat.venue_id → Venue.venue_id.
- **Event - offers - Ticket:** Ticket.event_id → Event.event_id.
- **Seat - assigned_to - Ticket:** Ticket.seat_id → Seat.seat_id.
- **User - joins - Waitlist:** Waitlist.user_id → User.user_id.
- **Waitlist - for - Event:** Waitlist.event_id → Event.event_id.

The ”Order contains Ticket” link is handled by a nullable foreign key Ticket.order_id (tickets exist before purchase; when sold, the Foreign Key is set). We use ”ON DELETE SET NULL” for Ticket.order_id and mostly ”ON DELETE CASCADE” elsewhere to keep inventory consistent when parents are removed.

Uniqueness & domain constraints

We enforce key domain rules directly in SQL:

- **Seat labels per venue:** UNIQUE(venue_id, seat_label).
- **One ticket per (event, seat):** UNIQUE(event_id, seat_id).
- **QR Codes:** UNIQUE(qr_code).

ISA hierarchies

We implemented all ISA hierarchies with the table per subclass pattern:

- **User** ⇒ **Customer**, **Organizer**: tables ”Customer(user_id FK)” and ”Organizer(user_id FK)” reuse the parent primary key as foreign key.
- **Event** ⇒ **Concert**, **Theatre**, **Sports**: each subtype table has event_id as primary key + foreign key.
- **Ticket** ⇒ **RegularTicket**, **PromoTicket**: each subtype has ticket_id as primary key + foreign key. Promos carry promo_code, discount.
- **Venue** ⇒ **IndoorVenue**, **OutdoorVenue**: each subtype has venue_id as primary key + foreign key.

This choice allows subtype-specific attributes later. We keep a ”User.role” enum for convenience. Consistency between ”role” and subclass membership will be enforced in application logic (an alternative is a trigger-based enforcement, which we omitted for portability).

Design alternatives (and why we chose this)

For ISA, we considered:

1. **Single table inheritance** (one table with a type column): simpler joins, but many nullable columns and weaker integrity.
2. **Table per subclass** (chosen): clean separation, strong integrity via foreign key to parent. Joins are acceptable for our workload.
3. **Table per concrete-class**: no parent table, but duplicates shared attributes and complicates cross-type queries.

For ”Order contains Ticket”, alternatives included a junction table. We opted for a nullable foreign key on Ticket to keep queries simple.

Testing

We weren't able to test the schema on CLAMV, due to not being able to access it. However, we did test it on a local MySQL instance using xampp. We created the tables, inserted sample data, and verified that constraints and relationships worked as intended.